

Mini RFC: Address Validation

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

Address validation ensures that an endpoint cannot be used for a traffic amplification attack. In such an attack, a packet is sent to a server with spoofed source address information that identifies a victim. If a server generates more or larger packets in response to that packet, the attacker can use the server to send more data toward the victim than it would be able to send on its own.

The primary defense against amplification attacks is verifying that a peer is able to receive packets at the transport address that it claims. Therefore, after receiving packets from an address that is not yet validated, an endpoint **MUST** limit the amount of data it sends to the unvalidated address to three times the amount of data received from that address. This limit on the size of responses is known as the anti-amplification limit.

Address validation is performed both during connection establishment and during connection migration.

Or a more general solution: Source Address Validation [RFC 7039].

2 Address Validation

Connection establishment implicitly provides address validation for both endpoints. In particular, receipt of a packet protected with Handshake keys confirms that the peer successfully processed an Initial packet. Once an endpoint has successfully processed a Handshake packet from the peer, it can consider the peer address to have been validated.

Additionally, an endpoint MAY consider the peer address validated if the peer uses a connection ID chosen by the endpoint and the connection ID contains at least 64 bits of entropy.

For the client, the value of the Destination Connection ID field in its first Initial packet allows it to validate the server address as a part of successfully processing any packet. Initial packets from the server are protected with keys that are derived from this value. Alternatively, the value is echoed by the server in Version Negotiation packets or included in the Integrity Tag in Retry packets.

Prior to validating the client address, servers MUST NOT send more than three times as many bytes as the number of bytes they have received. This limits the magnitude of any amplification attack that can be mounted using spoofed source addresses. For the purposes of avoiding amplification prior to address validation, servers MUST count all of the payload bytes received in datagrams that are uniquely attributed to a single connection. This includes datagrams that contain packets that are successfully processed and datagrams that contain packets that are all discarded.

Clients MUST ensure that UDP datagrams containing Initial packets have UDP payloads of at least 1200 bytes, adding PADDING frames as necessary. A client that sends padded datagrams allows the server to send more data prior to completing address validation.

Loss of an Initial or Handshake packet from the server can cause a deadlock if the client does not send additional Initial or Handshake packets. A deadlock could occur when the server reaches its anti-amplification limit and the client has received acknowledgments for all the data it has sent. In this case, when the client has no reason to send additional packets, the server will be unable to send more data because it has not validated the client's address. **To prevent this deadlock, clients MUST send a packet on a Probe Timeout (PTO).** Specifically, the client MUST send an Initial packet in a UDP datagram that contains at least 1200 bytes if it does not have Handshake keys, and otherwise send a Handshake packet.

Related optional feature: *Probe Timeout*

A server might wish to validate the client address before starting the cryptographic handshake. QUIC uses a token in the Initial packet to provide address validation prior to completing the handshake. This token is delivered to the client during connection establishment with a Retry packet or in a previous connection using the NEW_TOKEN frame.

In order to defend Initial flood attack similar as SYN flood attack.

In addition to sending limits imposed prior to address validation, servers are also constrained in what they can send by the limits set by the congestion controller. Clients are only constrained by the congestion controller.

And by the flow controller for stream data at both endpoints.

2.1 Address Validation Using Retry Packets

A token sent in a NEW_TOKEN frame or a Retry packet MUST be constructed in a way that allows the server to identify how it was provided to a client. These tokens are carried in the same field but require different handling from servers.

Consider SYN Cookies, an implementation may want to delay resource allocation until the peer is validated.

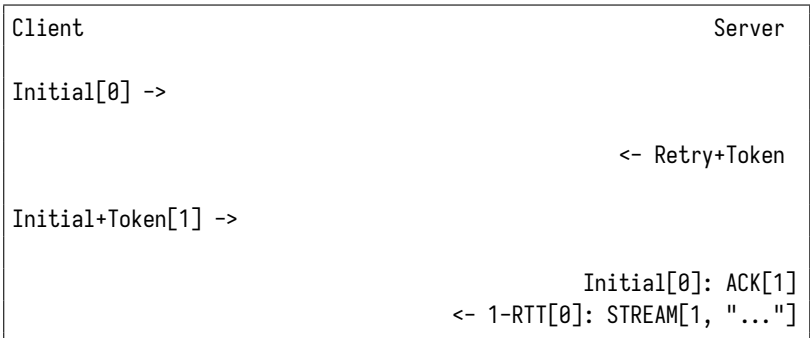
Upon receiving the client's Initial packet, the server can request address validation by sending a Retry packet containing a token. This token MUST be repeated by the client in all Initial packets it sends for that connection after it receives the Retry packet.

In response to processing an Initial packet containing a token that was provided in a Retry packet, a server cannot send another Retry packet; it can only refuse the connection or permit it to proceed.

As long as it is not possible for an attacker to generate a valid token for its own address and the client is able to return that token, it proves to the server that it received the token.

A server can also use a Retry packet to defer the state and processing costs of connection establishment. Requiring the server to provide a different connection ID, along with the original_destination_connection_id transport parameter, forces the server to demonstrate that it, or an entity it cooperates with, received the original Initial packet from the client. Providing a different connection ID also grants a server some control over how subsequent packets are routed. This can be used to direct connections to a different server instance.

If a server receives a client Initial that contains an invalid Retry token but is otherwise valid, it knows the client will not accept another Retry token. The server can discard such a packet and allow the client to time out to detect handshake failure, but that could impose a significant latency penalty on the client. Instead, the server SHOULD immediately close the connection with an INVALID_TOKEN error. Note that a server has not established any state for the connection at this point and so does not enter the closing period.



2.2 Address Validation for Future Connections

A server MAY provide clients with an address validation token during one connection that can be used on a subsequent connection. Address validation is especially important with 0-RTT because a server potentially sends a significant amount of data to a client in response to 0-RTT data.

The server uses the NEW_TOKEN frame to provide the client with an address validation token that can be used to validate future connections. In a

future connection, the client includes this token in Initial packets to provide address validation. The client **MUST** include the token in all Initial packets it sends, unless a Retry replaces the token with a newer one. The client **MUST NOT** use the token provided in a Retry for future connections. Servers **MAY** discard any Initial packet that does not carry the expected token.

Unlike the token that is created for a Retry packet, which is used immediately, the token sent in the NEW_TOKEN frame can be used after some period of time has passed. Thus, a token **SHOULD** have an expiration time, which could be either an explicit expiration time or an issued timestamp that can be used to dynamically calculate the expiration time. A server can store the expiration time or include it in an encrypted form in the token.

A token issued with NEW_TOKEN **MUST NOT** include information that would allow values to be linked by an observer to the connection on which it was issued. For example, it cannot include the previous connection ID or addressing information, unless the values are encrypted. A server **MUST** ensure that every NEW_TOKEN frame it sends is unique across all clients, with the exception of those sent to repair losses of previously sent NEW_TOKEN frames. Information that allows the server to distinguish between tokens from Retry and NEW_TOKEN **MAY** be accessible to entities other than the server.

It is unlikely that the client port number is the same on two different connections; validating the port is therefore unlikely to be successful.

A token received in a NEW_TOKEN frame is applicable to any server that the connection is considered authoritative for (e.g., server names included in the certificate). When connecting to a server for which the client retains an applicable and unused token, it **SHOULD** include that token in the Token field of its Initial packet. Including a token might allow the server to validate the client address without an additional round trip. A client **MUST NOT** include a token that is not applicable to the server that it is connecting to, unless the client has the knowledge that the server that issued the token and the server the client is connecting to are jointly managing the tokens. A client **MAY** use a token from any previous connection to that server.

A token allows a server to correlate activity between the connection where the token was issued and any connection where it is used. Clients that want to break continuity of identity with a server can discard tokens provided using the NEW_TOKEN frame. In comparison, a token obtained in a Retry packet **MUST** be used immediately during the connection attempt and cannot be used in subsequent connection attempts.

A client **SHOULD NOT** reuse a token from a NEW_TOKEN frame for different connection attempts. Reusing a token allows connections to be linked by entities on the network path.

Clients might receive multiple tokens on a single connection. Aside from preventing linkability, any token can be used in any connection attempt. Servers can send additional tokens to either enable address validation for multiple connection attempts or replace older tokens that might become invalid. For a client, this ambiguity means that sending the most

recent unused token is most likely to be effective. Though saving and using older tokens have no negative consequences, clients can regard older tokens as being less likely to be useful to the server for address validation.

When a server receives an Initial packet with an address validation token, it **MUST** attempt to validate the token, unless it has already completed address validation. If the token is invalid, then the server **SHOULD** proceed as if the client did not have a validated address, including potentially sending a Retry packet. Tokens provided with **NEW_TOKEN** frames and Retry packets can be distinguished by servers, and the latter can be validated more strictly. If the validation succeeds, the server **SHOULD** then allow the handshake to proceed.

The rationale for treating the client as unvalidated rather than discarding the packet is that the client might have received the token in a previous connection using the **NEW_TOKEN** frame, and if the server has lost state, it might be unable to validate the token at all, leading to connection failure if the packet is discarded.

In a stateless design, a server can use encrypted and authenticated tokens to pass information to clients that the server can later recover and use to validate a client address. Tokens are not integrated into the cryptographic handshake, and so they are not authenticated. For instance, a client might be able to reuse a token. To avoid attacks that exploit this property, a server can limit its use of tokens to only the information needed to validate client addresses.

Clients **MAY** use tokens obtained on one connection for any connection attempt using the same version. When selecting a token to use, clients do not need to consider other properties of the connection that is being attempted, including the choice of possible application protocols, session tickets, or other connection properties.

2.3 Address Validation Token Integrity

An address validation token **MUST** be difficult to guess. Including a random value with at least 128 bits of entropy in the token would be sufficient, but this depends on the server remembering the value it sends to clients.

A token-based scheme allows the server to offload any state associated with validation to the client. For this design to work, the token **MUST** be covered by integrity protection against modification or falsification by clients. Without integrity protection, malicious clients could generate or guess values for tokens that would be accepted by the server. Only the server requires access to the integrity protection key for tokens.

There is no need for a single well-defined format for the token because the server that generates the token also consumes it. Tokens sent in Retry packets **SHOULD** include information that allows the server to verify that the source IP address and port in client packets remain constant.

Tokens sent in **NEW_TOKEN** frames **MUST** include information that allows the server to verify that the client IP address has not changed from when

Notice the difference between tokens in Retry packets and **NEW_TOKEN** frames: the former needs to validate both IP address and port.

the token was issued. Servers can use tokens from NEW_TOKEN frames in deciding not to send a Retry packet, even if the client address has changed. If the client IP address has changed, the server MUST adhere to the anti-amplification limit. Note that in the presence of NAT, this requirement might be insufficient to protect other hosts that share the NAT from amplification attacks.

Attackers could replay tokens to use servers as amplifiers in DDoS attacks. To protect against such attacks, servers MUST ensure that replay of tokens is prevented or limited. Servers SHOULD ensure that tokens sent in Retry packets are only accepted for a short time, as they are returned immediately by clients. Tokens that are provided in NEW_TOKEN frames need to be valid for longer but SHOULD NOT be accepted multiple times. Servers are encouraged to allow tokens to be used only once, if possible; tokens MAY include additional information about clients to further narrow applicability or reuse.

2.4 Connection ID Negotiation with Retry packet

When the handshake includes a Retry, the negotiation includes one more step as shown in [Figure 1](#).

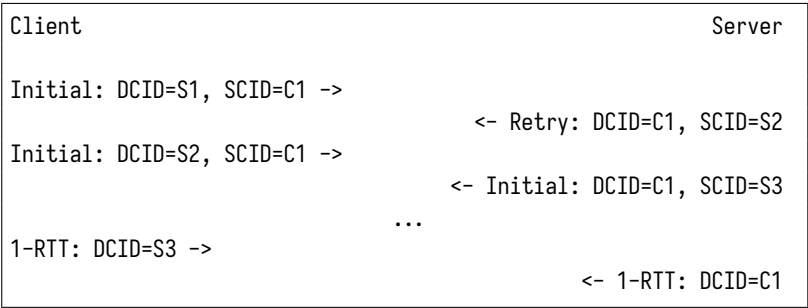


Figure 1: *Use of Connection IDs in a Handshake with Retry*

3 Packetization

3.1 Packet

3.1.1 Retry Packet

Retry packet uses a long packet header with a type value of 0x03. It carries an address validation token created by the server. It is used by a server that wishes to perform a retry.

Retry Packet { Header Form (1) = 1,
--

```

Fixed Bit (1) = 1,
Long Packet Type (2) = 3,
Unused (4),
Version (32),
Destination Connection ID Length (8),
Destination Connection ID (0..160),
Source Connection ID Length (8),
Source Connection ID (0..160),
Retry Token (..),
Retry Integrity Tag (128),
}

```

A Retry packet does not contain any protected fields. The value in the Unused field is set to an arbitrary value by the server; a client **MUST** ignore these bits. In addition to the fields from the long header, it contains these additional fields:

- **Retry Token:** An opaque token that the server can use to validate the client's address.
- **Retry Integrity Tag:** Omit if packet protection is not implemented.

The server populates the Destination Connection ID with the connection ID that the client included in the Source Connection ID of the Initial packet.

The server includes a connection ID of its choice in the Source Connection ID field. This value **MUST NOT** be equal to the Destination Connection ID field of the packet sent by the client. A client **MUST** discard a Retry packet that contains a Source Connection ID field that is identical to the Destination Connection ID field of its Initial packet. The client **MUST** use the value from the Source Connection ID field of the Retry packet in the Destination Connection ID field of subsequent packets that it sends.

A server **MAY** send Retry packets in response to Initial and 0-RTT packets. A server can either discard or buffer 0-RTT packets that it receives. A server can send multiple Retry packets as it receives Initial or 0-RTT packets. A server **MUST NOT** send more than one Retry packet in response to a single UDP datagram.

A client **MUST** accept and process at most one Retry packet for each connection attempt. After the client has received and processed an Initial or Retry packet from the server, it **MUST** discard any subsequent Retry packets that it receives.

Clients **MUST** discard Retry packets that have a Retry Integrity Tag that cannot be validated. This diminishes an attacker's ability to inject a Retry packet and protects against accidental corruption of Retry packets. A client **MUST** discard a Retry packet with a zero-length Retry Token field.

The client responds to a Retry packet with an Initial packet that includes the provided Retry token to continue connection establishment.

A client sets the Destination Connection ID field of this Initial packet to the value from the Source Connection ID field in the Retry packet. **Changing the Destination Connection ID field also results in a change to the keys used to protect the Initial packet.** It also sets the Token field to the token provided in the Retry packet. The client **MUST NOT** change the Source Connection ID because the server could include the connection ID as part of its token validation logic.

A Retry packet does not include a packet number and cannot be explicitly acknowledged by a client.

Subsequent Initial packets from the client include the connection ID and token values from the Retry packet. The client copies the Source Connection ID field from the Retry packet to the Destination Connection ID field and uses this value until an Initial packet with an updated value is received. The value of the Token field is copied to all subsequent Initial packets.

Other than updating the Destination Connection ID and Token fields, the Initial packet sent by the client is subject to the same restrictions as the first Initial packet. A client **MUST** use the same cryptographic handshake message it included in this packet. A server **MAY** treat a packet that contains a different cryptographic handshake message as a connection error or discard it. Note that including a Token field reduces the available space for the cryptographic handshake message, which might result in the client needing to send multiple Initial packets.

A client MAY attempt 0-RTT after receiving a Retry packet by sending 0-RTT packets to the connection ID provided by the server.

A client **MUST NOT** reset the packet number for any packet number space after processing a Retry packet. **In particular, 0-RTT packets contain confidential information that will most likely be retransmitted on receiving a Retry packet.** The keys used to protect these new 0-RTT packets will not change as a result of responding to a Retry packet. However, the data sent in these packets could be different than what was sent earlier. Sending these new packets with the same packet number is likely to compromise the packet protection for those packets because the same key and nonce could be used to protect different content. A server **MAY** abort the connection if it detects that the client reset the packet number.

The connection IDs used in Initial and Retry packets exchanged between client and server are copied to the transport parameters and validated.

4 Application Protocol

There is no new application protocol defined in this mini RFC.